

Chapter 5. SDK configuration

SDK configuration is the process of customising the SDK code for your particular build/application. As the parts of the SDK that you use are recompiled as part of your build, configuration options can be chosen at compile time resulting in smaller and more efficient customized versions of the code.

This chapter will show what configuration parameters are available, and how they can be changed.

SDK configuration parameters are passed as C preprocessor definitions to the build. The most common way to override them is to specify them in your `CMakeLists.txt` when you define your executable or library:

e.g.

```
add_executable(my_program main.c)
...
target_compile_definitions(my_program PRIVATE
    PICO_STACK_SIZE=4096
)
```

or if you are creating a library, and you want to add compile definitions whenever your library is included:

```
add_library(my_library INTERFACE)
...
target_compile_definitions(my_library INTERFACE
    PICO_STDIO_DEFAULT_CRLF=0
    PICO_DEFAULT_UART=1
)
```

The definitions can also be specified in header files, as is commonly done for board configuration (see [Chapter 8](#)).

For example, the Pimoroni Tiny2040 board header configures the following to specify appropriate board settings for the default I2C channel exposed on that board.

```
// --- I2C ---
#ifndef PICO_DEFAULT_I2C
#define PICO_DEFAULT_I2C 1
#endif
#ifndef PICO_DEFAULT_I2C_SDA_PIN
#define PICO_DEFAULT_I2C_SDA_PIN 2
#endif
#ifndef PICO_DEFAULT_I2C_SCL_PIN
#define PICO_DEFAULT_I2C_SCL_PIN 3
#endif
```

NOTE

The `#ifndef` still allows the values to be by the build (i.e. in `CMakeLists.txt`)

If you would rather set values in your own header file rather than via `CMake`, then you must make sure the header is included by all compilation (including the SDK sources). Using a custom `PICO_BOARD` header is one way of doing this, but a more advanced way is to have the SDK include your header via `pico/config.h` which itself is included by every SDK source file.

This can be done by adding the following before the `pico_sdk_init()` in your `CMakeLists.txt`:

```
list(APPEND PICO_CONFIG_HEADER_FILES path/to/your/header.h)
```

5.1. Full List of SDK Configuration Defines

Table 34. SDK and Board Configuration Defines

Name / Description	Default
CYW43_DEFAULT_PIN_WL_CLOCK gpio pin for the spi clock line to the cyw43 chip	
CYW43_DEFAULT_PIN_WL_CS gpio pin for the spi chip select to the cyw43 chip	
CYW43_DEFAULT_PIN_WL_DATA_IN gpio pin for spi data in from the cyw43 chip	
CYW43_DEFAULT_PIN_WL_DATA_OUT gpio pin for spi data out to the cyw43 chip	
CYW43_DEFAULT_PIN_WL_HOST_WAKE gpio (irq) pin for the irq line from the cyw43 chip	
CYW43_DEFAULT_PIN_WL_REG_ON gpio pin to power up the cyw43 chip	
CYW43_PIN_WL_DYNAMIC flag to indicate if cyw43 SPI pins can be changed at runtime	
CYW43_PIO_CLOCK_DIV_DYNAMIC Enable runtime configuration of the clock divider for communication with the wireless chip	0
CYW43_PIO_CLOCK_DIV_FRAC8 Fractional part of the clock divider for communication with the wireless chip 0-255	0
CYW43_PIO_CLOCK_DIV_INT Integer part of the clock divider for communication with the wireless chip	2
CYW43_TASK_PRIORITY Priority for the CYW43 FreeRTOS task	tskIDLE_PRIORITY + 4
CYW43_TASK_STACK_SIZE Stack size for the CYW43 FreeRTOS task in 4-byte words	1024
GPIO_IRQ_CALLBACK_ORDER_PRIORITY IRQ priority order of the default IRQ callback	PICO_SHARED_IRQ_HANDLER_LOWEST_ORDER_PRIORITY
GPIO_RAW_IRQ_HANDLER_DEFAULT_ORDER_PRIORITY IRQ priority order of raw IRQ handlers if the priority is not specified	PICO_SHARED_IRQ_HANDLER_DEFAULT_ORDER_PRIORITY
PARAM_ASSERTIONS_DISABLE_ALL Global assert disable	0
PARAM_ASSERTIONS_ENABLED_ADDRESS_ALIAS Enable/disable assertions in memory address aliasing macros	0
PARAM_ASSERTIONS_ENABLED_HARDWARE_ADC Enable/disable assertions in the hardware_adc module	0

Name / Description	Default
PARAM_ASSERTIONS_ENABLED_HARDWARE_BOOT_LOCK Enable/disable assertions in the hardware_boot_lock module	0
PARAM_ASSERTIONS_ENABLED_HARDWARE_CLOCKS Enable/disable assertions in the hardware_clocks module	0
PARAM_ASSERTIONS_ENABLED_HARDWARE_DMA Enable/disable hardware_dma assertions	0
PARAM_ASSERTIONS_ENABLED_HARDWARE_EXCEPTION Enable/disable assertions in the hardware_exception module	0
PARAM_ASSERTIONS_ENABLED_HARDWARE_FLASH Enable/disable assertions in the hardware_flash module	0
PARAM_ASSERTIONS_ENABLED_HARDWARE_GPIO Enable/disable assertions in the hardware_gpio module	0
PARAM_ASSERTIONS_ENABLED_HARDWARE_I2C Enable/disable assertions in the hardware_i2c module	0
PARAM_ASSERTIONS_ENABLED_HARDWARE_INTERP Enable/disable assertions in the hardware_interp module	0
PARAM_ASSERTIONS_ENABLED_HARDWARE_IRQ Enable/disable assertions in the hardware_irq module	0
PARAM_ASSERTIONS_ENABLED_HARDWARE_PIO Enable/disable assertions in the hardware_pio module	0
PARAM_ASSERTIONS_ENABLED_HARDWARE_POWMAN Enable/disable hardware_powman assertions	0
PARAM_ASSERTIONS_ENABLED_HARDWARE_PWM Enable/disable assertions in the hardware_pwm module	0
PARAM_ASSERTIONS_ENABLED_HARDWARE_RESETS Enable/disable assertions in the hardware_resets module	0
PARAM_ASSERTIONS_ENABLED_HARDWARE_SHA256 Enable/disable hardware_sha256 assertions	0
PARAM_ASSERTIONS_ENABLED_HARDWARE_SPI Enable/disable assertions in the hardware_spi module	0
PARAM_ASSERTIONS_ENABLED_HARDWARE_SYNC Enable/disable assertions in the hardware_sync module	0
PARAM_ASSERTIONS_ENABLED_HARDWARE_TICKS Enable/disable assertions in the hardware_ticks module	0
PARAM_ASSERTIONS_ENABLED_HARDWARE_TIMER Enable/disable assertions in the hardware_timer module	0
PARAM_ASSERTIONS_ENABLED_HARDWARE_UART Enable/disable assertions in the hardware_uart module	0
PARAM_ASSERTIONS_ENABLED_HARDWARE_WATCHDOG Enable/disable assertions in the hardware_watchdog module	0
PARAM_ASSERTIONS_ENABLED_HARDWARE_XIP_CACHE Enable/disable assertions in the hardware_xip_cache module	0

Name / Description	Default
PARAM_ASSERTIONS_ENABLED_LOCK_CORE Enable/disable assertions in the lock core	0
PARAM_ASSERTIONS_ENABLED_PHEAP Enable/disable assertions in the pheap module	0
PARAM_ASSERTIONS_ENABLED_PICO_CYW43_ARCH Enable/disable assertions in the pico_cyw43_arch module	0
PARAM_ASSERTIONS_ENABLED_PICO_MULTICORE Enable/disable assertions in the pico_multicore module	0
PARAM_ASSERTIONS_ENABLED_PICO_TIME Enable/disable assertions in the pico_time module	0
PARAM_ASSERTIONS_ENABLED_PIO_INSTRUCTIONS Enable/disable assertions in the PIO instructions	0
PARAM_ASSERTIONS_ENABLE_ALL Global assert enable	0
PICO_ADC_CLKDIV_ROUND_NEAREST True if floating point ADC clock divisors should be rounded to the nearest possible clock divisor rather than rounding down	PICO_CLKDIV_ROUND_NEAREST
PICO_BOOTROM_LOCKING_ENABLED Enable/disable locking for bootrom functions that use shared resources. If this flag is enabled bootrom lock checking is turned on and BOOT locks are taken around the relevant bootrom functions	1
PICO_BOOTSEL_VIA_DOUBLE_RESET_ACTIVITY_LED Optionally define a pin to use as bootloader activity LED when BOOTSEL mode is entered via reset double tap	
PICO_BOOTSEL_VIA_DOUBLE_RESET_ACTIVITY_LED_ACTIVE_LOW Whether pin used as bootloader activity LED when BOOTSEL mode is entered via reset double tap is active low. Not supported on RP2040	0
PICO_BOOTSEL_VIA_DOUBLE_RESET_INTERFACE_DISABLE_MASK Optionally disable either the mass storage interface (bit 0) or the PICOBOT interface (bit 1) when entering BOOTSEL mode via double reset	0
PICO_BOOTSEL_VIA_DOUBLE_RESET_TIMEOUT_MS Window of opportunity for a second press of a reset button to enter BOOTSEL mode (milliseconds)	200
PICO_BOOT_STAGE2_CHOOSE_AT25SF128A Select boot2_at25sf128a as the boot stage 2 when no boot stage 2 selection is made by the CMake build	0
PICO_BOOT_STAGE2_CHOOSE_GENERIC_03H Select boot2_generic_03h as the boot stage 2 when no boot stage 2 selection is made by the CMake build	1
PICO_BOOT_STAGE2_CHOOSE_IS25LP080 Select boot2_is25lp080 as the boot stage 2 when no boot stage 2 selection is made by the CMake build	0
PICO_BOOT_STAGE2_CHOOSE_W25Q080 Select boot2_w25q080 as the boot stage 2 when no boot stage 2 selection is made by the CMake build	0

Name / Description	Default
PICO_BOOT_STAGE2_CHOOSE_W25X10CL Select boot2_w25x10cl as the boot stage 2 when no boot stage 2 selection is made by the CMake build	0
PICO_BTSTACK_CYW43_MAX_HCI_PROCESS_LOOP_COUNT limit the max number of iterations of the hci processing loop	
PICO_BUILD_BOOT_STAGE2_NAME Name of the boot stage 2 if selected in the build system	
PICO_CLKDIV_ROUND_NEAREST True if floating point clock divisors should be rounded to the nearest possible clock divisor by default rather than rounding down	1
PICO_CLOCK_ADJUST_PERI_CLOCK_WITH_SYS_CLOCK When the SYS clock PLL is changed keep the peripheral clock attached to it	0
PICO_CLOCK_GPIO_CLKDIV_ROUND_NEAREST True if floating point GPIO clock divisors should be rounded to the nearest possible clock divisor rather than rounding down	PICO_CLKDIV_ROUND_NEAREST
PICO_CMSIS_RENAME_EXCEPTIONS Whether to rename SDK exceptions such as isr_nmi to their CMSIS equivalent i.e. NMI_Handler	1
PICO_CONFIG_HEADER Unquoted path to header include in place of the default pico/config.h which may be desirable for build systems which can't easily generate the config_autogen header	
PICO_CONFIG_RTOS_ADAPTER_HEADER Unquoted path to header include in the default pico/config.h for RTOS integration defines that must be included in all sources	
PICO_CORE1_STACK_SIZE Minimum amount of stack space reserved in the linker script for core 1 - if zero then no space is reserved and the user must provide their own stack	PICO_STACK_SIZE (0x800)
PICO_CYW43_ARCH_DEBUG_ENABLED Enable/disable some debugging output in the pico_cyw43_arch module	1 in debug builds
PICO_CYW43_ARCH_DEFAULT_COUNTRY_CODE Default country code for the cyw43 wireless driver	CYW43_COUNTRY_WORLDWIDE
PICO_DEBUG_MALLOC Enable/disable debug printf from malloc	0
PICO_DEBUG_MALLOC_LOW_WATER Define the lower bound for allocation addresses to be printed by PICO_DEBUG_MALLOC	0
PICO_DEBUG_PIN_BASE First pin to use for debug output (if enabled)	19
PICO_DEBUG_PIN_COUNT Number of pins to use for debug output (if enabled)	3
PICO_DEFAULT_I2C Define the default I2C for a board	Usually provided via board header
PICO_DEFAULT_I2C_SCL_PIN Define the default I2C SCL pin	Usually provided via board header

Name / Description	Default
PICO_DEFAULT_I2C_SDA_PIN Define the default I2C SDA pin	Usually provided via board header
PICO_DEFAULT_IRQ_PRIORITY Define the default IRQ priority	0x80
PICO_DEFAULT_LED_PIN Optionally define a pin that drives a regular LED on the board	Usually provided via board header
PICO_DEFAULT_LED_PIN_INVERTED 1 if LED is inverted or 0 if not	0
PICO_DEFAULT_SPI Define the default SPI for a board	Usually provided via board header
PICO_DEFAULT_SPI_CSN_PIN Define the default SPI CSN pin	Usually provided via board header
PICO_DEFAULT_SPI_RX_PIN Define the default SPI RX pin	Usually provided via board header
PICO_DEFAULT_SPI_SCK_PIN Define the default SPI SCK pin	Usually provided via board header
PICO_DEFAULT_SPI_TX_PIN Define the default SPI TX pin	Usually provided via board header
PICO_DEFAULT_TIMER Timer instance number to use for RP2040-period hardware_timer APIs that assumed a single timer instance	0
PICO_DEFAULT_UART Define the default UART used for printf etc	Usually provided via board header
PICO_DEFAULT_UART_BAUD_RATE Define the default UART baudrate	115200
PICO_DEFAULT_UART_RX_PIN Define the default UART RX pin	Usually provided via board header
PICO_DEFAULT_UART_TX_PIN Define the default UART TX pin	Usually provided via board header
PICO_DEFAULT_WS2812_PIN Optionally define a pin that controls data to a WS2812 compatible LED on the board	
PICO_DEFAULT_WS2812_POWER_PIN Optionally define a pin that controls power to a WS2812 compatible LED on the board	
PICO_DISABLE_SHARED_IRQ_HANDLERS Disable shared IRQ handlers	0
PICO_DOUBLE_SUPPORT_ROM_V1 RP2040 Include double support code for RP2040 B0 when that chip revision is supported	1
PICO_EMBED_XIP_SETUP Embed custom XIP setup (boot2) in an RP2350 binary	0

Name / Description	Default
PICO_FLASH_ASSERT_ON_UNSAFE Assert in debug mode rather than returning an error if flash_safe_execute cannot guarantee safety to catch bugs early	1
PICO_FLASH_ASSUME_CORE0_SAFE Assume that core 0 will never be accessing flash and so doesn't need to be considered during flash_safe_execute	0
PICO_FLASH_ASSUME_CORE1_SAFE Assume that core 1 will never be accessing flash and so doesn't need to be considered during flash_safe_execute	0
PICO_FLASH_BANK_STORAGE_OFFSET Offset in flash of the Bluetooth flash storage	PICO_FLASH_SIZE_BYTES - PICO_FLASH_BANK_TOTAL_SIZE
PICO_FLASH_BANK_TOTAL_SIZE Total size of the Bluetooth flash storage. Must be an even multiple of FLASH_SECTOR_SIZE	FLASH_SECTOR_SIZE * 2
PICO_FLASH_SAFE_EXECUTE_PICO_SUPPORT_MULTICORE_LOCKOUT Support using multicore_lockout functions to make the other core safe during flash_safe_execute	1 when using pico_multicore
PICO_FLASH_SAFE_EXECUTE_SUPPORT_FREERTOS_SMP Support using FreeRTOS SMP to make the other core safe during flash_safe_execute	1 when using FreeRTOS SMP
PICO_FLASH_SIZE_BYTES size of primary flash in bytes	Usually provided via board header
PICO_FLOAT_SUPPORT_ROM_V1 RP2040 Include float support code for RP2040 B0 when that chip revision is supported	1
PICO_HEAP_SIZE Minimum amount of heap space reserved by the linker script	0x800
PICO_INCLUDE_RTC_DATETIME Whether to include the datetime_t type used with the RP2040 RTC hardware	1 on RP2040
PICO_MALLOC_PANIC Enable/disable panic when an allocation failure occurs	1
PICO_MAX_SHARED_IRQ_HANDLERS Maximum number of shared IRQ handlers	4
PICO_MBEDTLS_SHA256_ALT_USE_DMA Whether to use DMA for writing to hardware for the mbedtls SHA-256 hardware acceleration	1
PICO_NO_BINARY_INFO Don't include "binary info" in the output binary	0 except for PICO_PLATFORM host
PICO_NO_FPGA_CHECK Remove the FPGA platform check for small code size reduction	1
PICO_NO_RAM_VECTOR_TABLE Enable/disable the RAM vector table	0
PICO_NO_SIM_CHECK RP2350 Remove the SIM platform check for small code size reduction	1

Name / Description	Default
PICO_OPAQUE_ABSOLUTE_TIME_T Enable opaque type for absolute_time_t to help catch inadvertent confusing uint64_t delays with absolute times	0
PICO_PANIC_FUNCTION Name of a function to use in place of the stock panic function or empty string to simply breakpoint on panic	
PICO_PHEAP_MAX_ENTRIES Maximum number of entries in the pheap	255
PICO_PIO_CLKDIV_ROUND_NEAREST True if floating point PIO clock divisors should be rounded to the nearest possible clock divisor rather than rounding down	PICO_CLKDIV_ROUND_NEAREST
PICO_PIO_USE_GPIO_BASE Enable code for handling more than 32 PIO pins	true when supported and when the device has more than 32 pins
PICO_PIO_VERSION PIO hardware version	0 on RP2040 and 1 on RP2350
PICO_PRINTF_ALWAYS_INCLUDED Whether to always include printf code even if only called weakly (by panic)	1 in debug build 0 otherwise
PICO_PRINTF_DEFAULT_FLOAT_PRECISION Define default floating point precision	6
PICO_PRINTF_FTOA_BUFFER_SIZE Define printf ftoa buffer size	32
PICO_PRINTF_MAX_FLOAT Define the largest float suitable to print with %f	1e9
PICO_PRINTF_NTOA_BUFFER_SIZE Define printf ntoa buffer size	32
PICO_PRINTF_SUPPORT_EXPONENTIAL Enable exponential floating point printing	1
PICO_PRINTF_SUPPORT_FLOAT Enable floating point printing	1
PICO_PRINTF_SUPPORT_LONG_LONG Enable support for long long types (%llu or %p)	1
PICO_PRINTF_SUPPORT_PTRDIFF_T Enable support for the ptrdiff_t type (%t)	1
PICO_PWM_CLKDIV_ROUND_NEAREST True if floating point PWM clock divisors should be rounded to the nearest possible clock divisor rather than rounding down	PICO_CLKDIV_ROUND_NEAREST
PICO_QUEUE_MAX_LEVEL Maintain a field for the highest level that has been reached by a queue	0
PICO_RAND_BUS_PERF_COUNTER_EVENT Bus performance counter event to use for sourcing entropy	arbiter_sram5_perf_event_access
PICO_RAND_BUS_PERF_COUNTER_INDEX Bus performance counter index to use for sourcing entropy	Undefined meaning pick one that is not counting any valid event already
PICO_RAND_ENTROPY_SRC_BUS_PERF_COUNTER Enable/disable use of a bus performance counter as an entropy source	1 if no hardware TRNG

Name / Description	Default
PICO_RAND_ENTROPY_SRC_ROSC Enable/disable use of ROSC as an entropy source	1 if no hardware TRNG
PICO_RAND_ENTROPY_SRC_TIME Enable/disable use of hardware timestamp as an entropy source	1
PICO_RAND_ENTROPY_SRC_TRNG Enable/disable use of hardware TRNG as an entropy source	1 if hardware TRNG is available
PICO_RAND_MIN_ROSC_BIT_SAMPLE_TIME_US Define a default minimum time between sampling the ROSC random bit	10
PICO_RAND_RAM_HASH_END End of address in RAM (non-inclusive) to hash during pico_rand seed initialization	SRAM_END
PICO_RAND_RAM_HASH_START Start of address in RAM (inclusive) to hash during pico_rand seed initialization	PICO_RAND_RAM_HASH_END - 1024
PICO_RAND_ROSC_BIT_SAMPLE_COUNT Number of samples to take of the ROSC random bit per random number generation	1
PICO_RAND_SEED_ENTROPY_SRC_BOARD_ID Enable/disable use of board id as part of the random seed	not PICO_RAND_SEED_ENTROPY_SRC_BOOT_RANDOM
PICO_RAND_SEED_ENTROPY_SRC_BOOT_RANDOM Enable/disable use of the per boot random number as an entropy source for the random seed	0 on RP2040 which has none
PICO_RAND_SEED_ENTROPY_SRC_BUS_PERF_COUNTER Enable/disable use of a bus performance counter as an entropy source for the random seed	PICO_RAND_ENTROPY_SRC_BUS_PERF_COUNTER
PICO_RAND_SEED_ENTROPY_SRC_RAM_HASH Enable/disable use of a RAM hash as an entropy source for the random seed	1 if no hardware TRNG
PICO_RAND_SEED_ENTROPY_SRC_ROSC Enable/disable use of ROSC as an entropy source for the random seed	PICO_RAND_ENTROPY_SRC_ROSC
PICO_RAND_SEED_ENTROPY_SRC_TIME Enable/disable use of hardware timestamp as an entropy source for the random seed	PICO_RAND_ENTROPY_SRC_TIME
PICO_RAND_SEED_ENTROPY_SRC_TRNG Enable/disable use of hardware TRNG as an entropy source for the random seed	PICO_RAND_ENTROPY_SRC_TRNG
PICO_RP2040_B0_SUPPORTED RP2040 Whether to include any specific software support for RP2040 B0 revision	1
PICO_RP2040_B1_SUPPORTED RP2040 Whether to include any specific software support for RP2040 B1 revision	1
PICO_RP2040_B2_SUPPORTED RP2040 Whether to include any specific software support for RP2040 B2 revision	1
PICO_RP2350A RP2350 Whether the current board has an RP2350 in an A (30 GPIO) package	Usually provided via board header
PICO_RP2350_A2_SUPPORTED RP2350 Whether to include any specific software support for RP2350 A2 revision	1

Name / Description	Default
PICO_RUNTIME_NO_INIT_BOOTROM_RESET Do not include SDK implementation of <code>runtime_init_bootrom_reset</code> function	1 on RP2040
PICO_RUNTIME_NO_INIT_CLOCKS Do not include SDK implementation of <code>runtime_init_clocks</code> function	0
PICO_RUNTIME_NO_INIT_DEFAULT_ALARM_POOL Do not include SDK implementation of <code>runtime_init_default_alarm_pool</code> function	1 if <code>PICO_TIME_DEFAULT_ALARM_POOL_DISABLED</code> is
PICO_RUNTIME_NO_INIT_EARLY_RESETS Do not include SDK implementation of <code>runtime_init_early_resets</code> function	1 on RP2040
PICO_RUNTIME_NO_INIT_INSTALL_RAM_VECTOR_TABLE Do not include SDK implementation of <code>runtime_init_install_ram_vector_table</code> function	0 unless RISC-V or RAM binary
PICO_RUNTIME_NO_INIT_MUTEX Do not include SDK implementation of <code>runtime_init_mutex</code> function	0
PICO_RUNTIME_NO_INIT_PER_CORE_BOOTROM_RESET Do not include SDK implementation of <code>runtime_init_per_core_bootrom_reset</code> function	1 on RP2040
PICO_RUNTIME_NO_INIT_PER_CORE_ENABLE_COPROCESSORS Do not include SDK implementation of <code>runtime_init_per_core_enable_coprocessors</code> function	1 on RP2040 or RISC-V
PICO_RUNTIME_NO_INIT_PER_CORE_INSTALL_STACK_GUARD Do not include SDK implementation of <code>runtime_init_per_core_install_stack_guard</code> function	1 unless <code>PICO_USE_STACK_GUARDS</code> is 1
PICO_RUNTIME_NO_INIT_RP2040_GPIO_IE_DISABLE RP2040 Do not include SDK implementation of <code>runtime_init_rp2040_gpio_ie_disable</code> function	0 on RP2040
PICO_RUNTIME_NO_INIT_SPIN_LOCKS_RESET Do not include SDK implementation of <code>runtime_init_spin_locks_reset</code> function	0
PICO_RUNTIME_NO_INIT_USB_POWER_DOWN Do not include SDK implementation of <code>runtime_init_usb_power_down</code> function	0
PICO_RUNTIME_SKIP_INIT_BOOTROM_LOCKING_ENABLE Skip calling of <code>runtime_init_bootrom_locking_enable</code> function during runtime init	0
PICO_RUNTIME_SKIP_INIT_BOOTROM_RESET Skip calling of <code>runtime_init_bootrom_reset</code> function during runtime init	1 on RP2040
PICO_RUNTIME_SKIP_INIT_BOOT_LOCKS_RESET Skip calling of <code>runtime_init_boot_locks_reset</code> function during runtime init	0
PICO_RUNTIME_SKIP_INIT_CLOCKS Skip calling of <code>runtime_init_clocks</code> function during runtime init	0
PICO_RUNTIME_SKIP_INIT_DEFAULT_ALARM_POOL Skip calling of <code>runtime_init_default_alarm_pool</code> function during runtime init	1 if <code>PICO_TIME_DEFAULT_ALARM_POOL_DISABLED</code> is 1
PICO_RUNTIME_SKIP_INIT_EARLY_RESETS Skip calling of <code>runtime_init_early_resets</code> function during runtime init	1 on RP2040

Name / Description	Default
PICO_RUNTIME_SKIP_INIT_INSTALL_RAM_VECTOR_TABLE Skip calling of <code>runtime_init_install_ram_vector_table</code> function during runtime init	0 unless RISC-V or RAM binary
PICO_RUNTIME_SKIP_INIT_MUTEX Skip calling of <code>runtime_init_mutex</code> function during runtime init	0
PICO_RUNTIME_SKIP_INIT_PER_CORE_BOOTROM_RESET Skip calling of <code>runtime_init_per_core_bootrom_reset</code> function during per-core init	1 on RP2040
PICO_RUNTIME_SKIP_INIT_PER_CORE_ENABLE_COPROCESSORS Skip calling of <code>runtime_init_per_core_enable_coprocessors</code> function during per-core init	1 on RP2040 or RISC-V
PICO_RUNTIME_SKIP_INIT_PER_CORE_H3_IRQ_REGISTERS Skip calling of <code>runtime_init_per_core_h3_irq_registers</code> function during per-core init	1 on non RISC-V
PICO_RUNTIME_SKIP_INIT_PER_CORE_INSTALL_STACK_GUARD Skip calling of <code>runtime_init_per_core_install_stack_guard</code> function during runtime init	1 unless <code>PICO_USE_STACK_GUARDS</code> is 1
PICO_RUNTIME_SKIP_INIT_RP2040_GPIO_IE_DISABLE RP2040 Skip calling of <code>runtime_init_rp2040_gpio_ie_disable</code> function during runtime init	0 on RP2040
PICO_RUNTIME_SKIP_INIT_SPIN_LOCKS_RESET Skip calling of <code>runtime_init_spin_locks_reset</code> function during runtime init	0
PICO_RUNTIME_SKIP_INIT_USB_POWER_DOWN Skip calling of <code>runtime_init_usb_power_down</code> function during runtime init	0
PICO_SHARED_IRQ_HANDLER_DEFAULT_ORDER_PRIORITY Set default shared IRQ order priority	0x80
PICO_SPINLOCK_ID_ATOMIC Spinlock ID for atomics	13
PICO_SPINLOCK_ID_CLAIM_FREE_FIRST Lowest Spinlock ID in the 'claim free' range	24
PICO_SPINLOCK_ID_CLAIM_FREE_LAST Highest Spinlock ID in the 'claim free' range	31
PICO_SPINLOCK_ID_HARDWARE_CLAIM Spinlock ID for Hardware claim protection	11
PICO_SPINLOCK_ID_IRQ Spinlock ID for IRQ protection	9
PICO_SPINLOCK_ID_OS1 First Spinlock ID reserved for use by low level OS style software	14
PICO_SPINLOCK_ID_OS2 Second Spinlock ID reserved for use by low level OS style software	15
PICO_SPINLOCK_ID_RAND Spinlock ID for Random Number Generator	12
PICO_SPINLOCK_ID_STRIPED_FIRST Lowest Spinlock ID in the 'striped' range	16
PICO_SPINLOCK_ID_STRIPED_LAST Highest Spinlock ID in the 'striped' range	23

Name / Description	Default
PICO_SPINLOCK_ID_TIMER Spinlock ID for Timer protection	10
PICO_STACK_SIZE Minimum amount of stack space reserved in the linker script for each core. See also PICO_CORE1_STACK_SIZE	0x800
PICO_STDIO_DEADLOCK_TIMEOUT_MS Time after which to assume stdio_usb is deadlocked by use in IRQ and give up	1000
PICO_STDIO_DEFAULT_CRLF Default for CR/LF conversion enabled on all stdio outputs	1
PICO_STDIO_ENABLE_CRLF_SUPPORT Enable/disable CR/LF output conversion support	1
PICO_STDIO_RTT_DEFAULT_CRLF Default state of CR/LF translation for rtt output	PICO_STDIO_DEFAULT_CRLF
PICO_STDIO_SEMIHOSTING_DEFAULT_CRLF Default state of CR/LF translation for semihosting output	PICO_STDIO_DEFAULT_CRLF
PICO_STDIO_SHORT_CIRCUIT_CLIB_FUNCS Directly replace common stdio functions such as putchar from the C-library to avoid pulling in lots of c library code for simple output	1
PICO_STDIO_STACK_BUFFER_SIZE Define printf buffer size (on stack)... this is just a working buffer not a max output size	128
PICO_STDIO_UART_DEFAULT_CRLF Default state of CR/LF translation for UART output	PICO_STDIO_DEFAULT_CRLF
PICO_STDIO_UART_SUPPORT_CHARS_AVAILABLE_CALLBACK Enable UART STDIO support for stdio_set_chars_available_callback. Can be disabled to make use of the uart elsewhere	1
PICO_STDIO_USB_CONNECTION_WITHOUT_DTR Disable use of DTR for connection checking meaning connection is assumed to be valid	0
PICO_STDIO_USB_CONNECT_WAIT_TIMEOUT_MS Maximum number of milliseconds to wait during initialization for a CDC connection from the host (negative means indefinite) during initialization	0
PICO_STDIO_USB_DEFAULT_CRLF Default state of CR/LF translation for USB output	PICO_STDIO_DEFAULT_CRLF
PICO_STDIO_USB_DEINIT_DELAY_MS Number of milliseconds to wait before deinitializing stdio_usb	110
PICO_STDIO_USB_DEVICE_SELF_POWERED Set USB device as self powered device	0
PICO_STDIO_USB_ENABLE_RESET_VIA_BAUD_RATE Enable/disable resetting into BOOTSEL mode if the host sets the baud rate to a magic value (PICO_STDIO_USB_RESET_MAGIC_BAUD_RATE)	1 if application is not using TinyUSB directly
PICO_STDIO_USB_ENABLE_RESET_VIA_VENDOR_INTERFACE Enable/disable resetting into BOOTSEL mode via an additional VENDOR USB interface - enables picotool based reset	1 if application is not using TinyUSB directly

Name / Description	Default
PICO_STDIO_USB_LOW_PRIORITY_IRQ Explicit User IRQ number to claim for tud_task() background execution instead of letting the implementation pick a free one dynamically (deprecated)	
PICO_STDIO_USB_POST_CONNECT_WAIT_DELAY_MS Number of extra milliseconds to wait when using PICO_STDIO_USB_CONNECT_WAIT_TIMEOUT_MS after a host CDC connection is detected (some host terminals seem to sometimes lose transmissions sent right after connection)	50
PICO_STDIO_USB_RESET_BOOTSEL_ACTIVITY_LED Optionally define a pin to use as bootloader activity LED when BOOTSEL mode is entered via USB (either VIA_BAUD_RATE or VIA_VENDOR_INTERFACE)	
PICO_STDIO_USB_RESET_BOOTSEL_ACTIVITY_LED_ACTIVE_LOW Whether pin to use as bootloader activity LED when BOOTSEL mode is entered via USB (either VIA_BAUD_RATE or VIA_VENDOR_INTERFACE) is active low	0
PICO_STDIO_USB_RESET_BOOTSEL_FIXED_ACTIVITY_LED Whether the pin specified by PICO_STDIO_USB_RESET_BOOTSEL_ACTIVITY_LED is fixed or can be modified by picotool over the VENDOR USB interface	0
PICO_STDIO_USB_RESET_BOOTSEL_INTERFACE_DISABLE_MASK Optionally disable either the mass storage interface (bit 0) or the PICOBOOT interface (bit 1) when entering BOOTSEL mode via USB (either VIA_BAUD_RATE or VIA_VENDOR_INTERFACE)	0
PICO_STDIO_USB_RESET_INTERFACE_SUPPORT_MS_OS_20_DESCRIPTOR If vendor reset interface is included add support for Microsoft OS 2.0 Descriptor	1
PICO_STDIO_USB_RESET_INTERFACE_SUPPORT_RESET_TO_BOOTSEL If vendor reset interface is included allow rebooting to BOOTSEL mode	1
PICO_STDIO_USB_RESET_INTERFACE_SUPPORT_RESET_TO_FLASH_BOOT If vendor reset interface is included allow rebooting with regular flash boot	1
PICO_STDIO_USB_RESET_MAGIC_BAUD_RATE Baud rate that if selected causes a reset into BOOTSEL mode (if PICO_STDIO_USB_ENABLE_RESET_VIA_BAUD_RATE is set)	1200
PICO_STDIO_USB_RESET_RESET_TO_FLASH_DELAY_MS Delay in ms before rebooting via regular flash boot	100
PICO_STDIO_USB_STDOUT_TIMEOUT_US Number of microseconds to be blocked trying to write USB output before assuming the host has disappeared and discarding data	500000
PICO_STDIO_USB_SUPPORT_CHARS_AVAILABLE_CALLBACK Enable USB STDIO support for stdio_set_chars_available_callback. Can be disabled to make use of USB CDC RX callback elsewhere	1
PICO_STDIO_USB_TASK_INTERVAL_US Period of microseconds between calling tud_task in the background	1000
PICO_STDOUT_MUTEX Enable/disable mutex around stdout	1
PICO_TIME_DEFAULT_ALARM_POOL_DISABLED Disable the default alarm pool	0

Name / Description	Default
PICO_TIME_DEFAULT_ALARM_POOL_HARDWARE_ALARM_NUM Select which HW alarm is used for the default alarm pool	3
PICO_TIME_DEFAULT_ALARM_POOL_MAX_TIMERS Selects the maximum number of concurrent timers in the default alarm pool	16
PICO_TIME_SLEEP_OVERHEAD_ADJUST_US How many microseconds to wake up early (and then busy_wait) to account for timer overhead when sleeping in low power mode	6
PICO_UART_DEFAULT_CRLF Enable/disable CR/LF translation on UART	0
PICO_UART_ENABLE_CRLF_SUPPORT Enable/disable CR/LF translation support	1
PICO_USE_FASTEST_SUPPORTED_CLOCK Use the fastest officially supported clock by default	0
PICO_USE_GPIO_COPROCESSOR Enable/disable use of the GPIO coprocessor for GPIO access	1
PICO_USE_MALLOC_MUTEX Whether to protect malloc etc with a mutex	1 with pico_multicore, 0 otherwise
PICO_USE_STACK_GUARDS RP2350 Enable/disable stack guards	0
PICO_USE_SW_SPIN_LOCKS Use software implementation for spin locks	1 on RP2350 due to errata
PICO_VTABLE_PER_CORE User is using separate vector tables per core	0
PICO_XOSC_STARTUP_DELAY_MULTIPLIER Multiplier (from 1ms) for xosc startup delay to accommodate slow-starting oscillators	6
PLL_SYS_POSTDIV1 System clock PLL post divider 1 setting	6 on RP2040 or 5 on RP2350
PLL_SYS_POSTDIV2 System clock PLL post divider 2 setting	2
PLL_SYS_REFDIV PLL reference divider setting for PLL_SYS	1
PLL_SYS_VCO_FREQ_HZ System clock PLL frequency	(1500 * MHZ)
PLL_USB_POSTDIV1 USB clock PLL post divider 1 setting	5
PLL_USB_POSTDIV2 USB clock PLL post divider 2 setting	5
PLL_USB_REFDIV PLL reference divider setting for PLL_USB	1
PLL_USB_VCO_FREQ_HZ USB clock PLL frequency	(1200 * MHZ)
SYS_CLK_HZ RP2040 System operating frequency in Hz	125000000

Name / Description	Default
SYS_CLK_HZ RP2350 System operating frequency in Hz	150000000
SYS_CLK_VREG_VOLTAGE_AUTO_ADJUST Should the regulator voltage be adjusted above SYS_CLK_VREG_VOLTAGE_MIN when initializing the clocks	0
SYS_CLK_VREG_VOLTAGE_AUTO_ADJUST_DELAY_US Number of microseconds to wait after updating regulator voltage due to SYS_CLK_VREG_VOLTAGE_MIN to allow voltage to settle	1000
SYS_CLK_VREG_VOLTAGE_MIN minimum voltage (see VREG_VOLTAGE_x_xx) for the voltage regulator to be ensured during clock initialization if SYS_CLK_VREG_VOLTAGE_AUTO_ADJUST is 1	
USB_CLK_HZ USB clock frequency. Must be 48MHz for the USB interface to operate correctly	48000000
USB_DPRAM_MAX Set amount of USB RAM used by USB system	4096
XOSC_HZ Crystal oscillator frequency in Hz	12000000